

NextRead

(Recommender System for Book-Crossing dataset)

Neelima Gurram

Department of Computer Science and Engineering
Wright State University, Dayton, OH
Email: gurram.60@wright.edu

Abstract:

This project uses the Book-Crossing dataset to develop a book recommendation system, experimenting with memory-based collaborative filtering techniques as well as matrix factorization with Singular Value Decomposition (SVD). Initially, memory-based collaborative filtering, such as cosine similarity, was used to compare users and recommend books based on their interests. However, SVD outperformed this method because of its capacity to efficiently reduce the dimensionality of the user-item interaction matrix while collecting latent features that describe underlying patterns in both user preferences and book attributes. By decomposing the matrix, SVD tackles the dataset's sparsity and enhances the accuracy of book suggestions. The model's performance is measured using Root Mean Square Error (RMSE), which demonstrates SVD's greater prediction accuracy. This project highlights the advantages of SVD over traditional memory-based methods in delivering more accurate and scalable book recommendations.

Keywords: Recommender System, Collaborative Filtering, Singular Value Decomposition(SVD), Book-Crossing Dataset, Matrix Factorization, Cosine Similarity, Memory-based Collaborative Filtering, Personalized Recommendations, Book Recommendations, Item-based Filtering, User-based Filtering, Model-based Collaborative Filtering, Latent Factors, Data Sparsity, User-Item Interaction Matrix, Dimensionality Reduction, Root Mean Square Error(RMSE).

1 Introduction

Recommender systems are frequently utilized in a variety of areas, including e-commerce, movies, music, and literature, to give consumers with individualized recommendations. The Book-Crossing dataset, which comprises user-generated book ratings, is an excellent starting point for developing a book recommendation system. Collaborative filtering techniques, such as memory-based and model-based approaches, are widely used to forecast user preferences and recommend appropriate items. While memory-based methods like cosine similarity look for similarities between persons or things, matrix factorization techniques like Singular Value Decomposition (SVD) can provide more efficient solutions by discovering hidden patterns in the data.

1.1 Motivation

With an ever-increasing amount of books and user-generated content, tailored book suggestions are crucial. Choosing new books can be daunting for users, so they rely on suggestions from reliable sources. The ability to recommend appropriate books based on a user's previous choices or those of similar users can greatly improve their reading experience. Although memory-based collaborative filtering algorithms produce acceptable results, their performance can suffer in the face of sparse data, making model-based approaches, such as SVD, a more promising option.

1.2 Problem Statement

A recommendation system helps an organization to create loyal customers and build trust by them desired products and services for which they came on your site. The recommendation system today is so powerful that they can handle the new customer too who has visited the site for the first time. They recommend the products which are currently trending or highly rated and they can also recommend the products which bring maximum profit to the company. Providing specific data analysis and prediction to done with this data. The main objective is to built a predictive recommender model, which could help in predicting – how we can predict the best recommendation for users according to their items approach. This would help us in providing better recommendation item to a right specific users.

1.3 Novelty

Unlike memory-based approaches, which compute similarity between individuals or things, SVD collects latent features that represent both user preferences and book attributes, hence lowering data dimensionality. By testing the model using Root Mean Square Error (RMSE), we want to show that SVD outperforms classic memory-based techniques like cosine similarity in terms of scalability and prediction accuracy. This approach is unique in that it can handle sparse data and deliver more accurate, individualized book suggestions, hence improving user satisfaction and discovery.

2 Dataset

2.1 Source

<https://www.kaggle.com/datasets/arashnic/book-recommendation-dataset?resource=download>

2.2 Data Statistics

- Book-Ratings.csv – Contains ratings given by users to books.
Number of users: 278,858
Number of books: 271,379
Number of ratings: 1,149,780
Ratings are on a scale from 1 to 10.
- Books.csv – Contains metadata about the books.
Features include: ISBN, title, author, year of publication, publisher, etc.
- Users.csv – Contains information about the users.
Features include: User ID, location, and age .

2.3 Features

The Book-Crossing dataset includes the following key features:

- **User ID:** A unique identifier for each user.
- **ISBN:** A unique identifier for each book.
- **Rating:** The rating given by a user to a specific book, ranging from 1 to 10.
- **Title, Author, and Publisher** (from the Books.csv file): Metadata about each book, which helps in categorizing and understanding the content.

2.4 Data Preprocessing

Beginning with book data – dropping URL features (i.e 'Image-URL-S', 'Image-URL-M', 'Image-URL-L'). The data has some extra columns which are not required for our task like image URLs. And I renamed the columns of each file as the name of the column contains space and lowercase letters so I corrected to make it easy to use. The features depict great analysis by feature engineering. Some of the null values were present in feature data, we replaced with mean of that particular feature. Deal with mismatch feature like book_title, book_author, year_of_publication, publisher. Only considering age between 5-90 I took users data to analysis and perform recommendation on it.

2.5 Data Split

To evaluate the performance of the recommendation system, the data was split into training and testing sets. Specifically:

- Training Set: 80% of the data (0.8 of the dataset) was used for training the model.
- Testing Set: 20% of the data (0.2 of the dataset) was reserved as the testing set to evaluate the accuracy of the model.

The data split was done randomly, ensuring that both the training and testing sets represent the overall distribution of ratings. This helps in assessing the model's ability to generalize to unseen data.

3 Methodology

This project explores two primary approaches for building a book recommendation system: Memory-Based Collaborative Filtering and Model-Based Collaborative Filtering. Both approaches utilize user-item interactions to make predictions, but they differ significantly in how they process and model the data.

3.1 Memory-Based Collaborative Filtering (User-based & Item-based)

The first approach tested in the project was **Memory-Based Collaborative Filtering**, which leverages the past interactions between users and items (books, in this case) to make recommendations. Memory-based filtering relies heavily on the concept of similarity between users or items to generate predictions.

- User-Based Collaborative Filtering:
In this approach, the system identifies users who have similar preferences based on the books they have rated. If two users have rated books similarly in the past, the system assumes they will like similar books in the future. The system recommends books to a user based on what similar users have liked.
- Item-Based Collaborative Filtering:
Instead of comparing users, this approach compares items (books). The system calculates the similarity between books based on the ratings given by users. If two books are rated similarly by the same group of users, the system assumes that if a user likes one book, they will also like the other. Recommendations are made by identifying items that are most similar to the books the user has rated highly.

In both cases, **Cosine Similarity** was used to calculate the similarity between users or items. Cosine similarity measures the cosine of the angle between two vectors representing users' or items' ratings. The formula for cosine similarity between two vectors A and B is:

$$\text{Cosine similarity} = \frac{A \cdot B}{\|A\| \|B\|} \quad (1)$$

Where:

- A and B are the rating vectors of two users or two items.
- $\|A\|$ and $\|B\|$ are the magnitudes of the vectors.

Once the similarities were computed, the system used these scores to predict ratings for items that a user had not yet rated, with higher scores suggesting stronger recommendations.

3.1.1 Challenges with Memory-Based Methods:

Memory-based approaches face challenges in handling **sparse datasets** (where many users haven't rated a large number of books). This sparsity can lead to less accurate predictions, particularly when users or items have few interactions. Additionally, memory-based methods can become computationally expensive as the number of users and items increases.

3.2 Model-Based Collaborative Filtering (SVD)

To address the limitations of memory-based methods, we transitioned to a **Model-Based Collaborative Filtering** approach using **Singular Value Decomposition (SVD)**. SVD is a powerful **latent factor model** that reduces the dimensionality of the user-item interaction matrix and helps uncover latent patterns in the data.

3.2.1 Latent Factor Model(SVD):

SVD is a **matrix factorization technique** that decomposes the large user-item interaction matrix R into three smaller matrices: U, Σ , and V, where:

- **U** is the matrix of user features.
- **Σ** is a diagonal matrix containing the singular values.
- **V** is the matrix of item features.

The decomposition is represented as:

$$R \approx U \cdot \Sigma \cdot (V^T) \quad (2)$$

The core idea behind SVD is that the user-item interaction matrix can be approximated using a smaller, low-rank approximation. This approximation captures the **latent factors** that explain the observed user-item interactions. These latent factors could represent underlying characteristics of books (e.g., genre, style) and users (e.g., reading preferences).

3.2.2 Dimensionality Reduction:

The rank of the matrices **U**, **Σ**, and **V** determines the number of latent factors. By selecting a smaller number of latent factors (reducing the dimensionality), we focus on the most important underlying patterns in the data, which helps reduce noise and overfitting. This is the key advantage of SVD over memory-based methods, as it can efficiently handle sparse data and provide more accurate predictions by focusing on the most meaningful features.

3.2.3 Predictions with SVD:

Once the decomposition is complete, predictions for missing ratings are made by multiplying the three matrices **U**, **Σ** and **V^T**. This approximation allows the system to fill in the missing ratings with predicted values based on the learned latent factors.

- Recommendations are generated by selecting the items with the highest predicted ratings for each user, provided they have not yet rated them.

3.3 Implementation

Collaborative Filtering - Memory based approach was my first trial on train and test dataset which uses the memory of previous users interactions to compute users similarities based on items they've interacted (i.e user-based approach) or compute items similarities based on the users that have interacted with them (i.e item-based approach). Applying cosine similarity to make item-item similarity need to take transpose of matrix. This matrix would help in manage train-test matrix. After all views predictions based on similarity, I try to find recommendation on it based on score. Model-based collaborative filtering algorithms provide item recommendation by first developing a model of user ratings. I used Latent Factor Model called Singular Value Decomposition (SVD). SVD made dimensionality reduction technique in machine learning. SVD is a matrix factorization technique. It made us reduces the number of features of a dataset by reducing the space dimension from N-dimension to K-dimension (where $K < N$). SVD uses a matrix structure where each row represents a user, and each column represents an item. The elements of this matrix are the ratings that are given to items by users.

4 Experimental Results

4.1 Evaluation Metrics

To assess the effectiveness of both collaborative filtering approaches, I used Root Mean Square Error (RMSE) as the evaluation metric. RMSE quantifies the difference between predicted ratings and actual ratings from the test data. Lower RMSE values indicate better predictive accuracy.

The RMSE is calculated as:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (r_{true,i} - r_{pred,i})^2} \quad (3)$$

Where:

- $r_{true,i}$ is the actual rating for the i -th user-item pair.
- $r_{pred,i}$ is the predicted rating for the i -th user-item pair.
- n is the number of predicted ratings.

RMSE Scores:

Table 1. Comparison of RMSE scores

Model	RMSE (Training)	RMSE (Testing)
User-Based Cosine Similarity	7.94	7.94
Item-Based Cosine Similarity	7.94	7.94
SVD (Singular Value Decomposition)	1.64	1.64

- The **User-Based** and **Item-Based Cosine Similarity** models both show an RMSE of **7.94**, indicating relatively high error in predicting user-item ratings.
- The **SVD model** performs significantly better, with an RMSE of **1.64**, demonstrating a large improvement in predictive accuracy due to the matrix factorization technique.

4.2 Results

[137]:

	uid	iid	r_ui	est	details
0	171082	1551666650	10.0	8.214163	{'was_impossible': False}
1	107784	0842338454	5.0	5.528890	{'was_impossible': False}
2	177458	0805022945	8.0	7.211443	{'was_impossible': False}
3	28591	0312303467	10.0	8.096651	{'was_impossible': False}
4	46909	0786014474	6.0	8.767632	{'was_impossible': False}
...	...	...	...	...	...
76764	98723	0684196395	8.0	7.656983	{'was_impossible': False}
76765	129008	0345438329	5.0	7.231967	{'was_impossible': False}
76766	170615	0385336195	7.0	7.498766	{'was_impossible': False}
76767	230776	0767904249	8.0	7.942021	{'was_impossible': False}
76768	14422	0316734837	8.0	7.577136	{'was_impossible': False}

76769 rows × 5 columns

Fig. 1. Final unique user-item interaction matrix obtained

```
[144]: #take a random look at user_id
example_reading_list = get_reading_list(userid=116866)
for book, rating in example_reading_list.items():
    print(f'{book}: {rating}')

Chaos: Making a New Science: 8.214803961490311
SHIPPING NEWS: 7.983060085516431
The Dinosaur Project: The Story of the Greatest Dinosaur Hunt Ever Mounted: 7.829945208060512
Landmarks in Western Science: From Prehistory to the Atomic Age: 7.829945208060512
The Complete Idiot's Guide to Javascript: 7.829945208060512
The Moscow Puzzles: 359 Mathematical Recreations: 7.829945208060512
Granta 52: Food : The Vital Stuff: 7.829945208060512
River Why: 7.829945208060512
Climbing Mount Improbable: 7.7972344227921635
The Man Who Tasted Shapes: A Bizarre Medical Mystery Offers Revolutionary Insights into Emotions, Reasoning, and Consciousness: 7.706972834149946

[146]: example_reading_list=get_reading_list(userid=274061)
for book, rating in example_reading_list.items():
    print(f'{book}: {rating}')

Savage Thunder: 10
Strangers: 10
Swan Song: 10
Silver on the Tree: 10
Charming the Prince: 10
Dark Desire (Paranormal Romance): 10
Riptide: 9.938293974510724
Magic Knight Rayearth 1: 9.811302980187254
The Magic of You (Malory Novels (Paperback)): 9.811069484563186
Clover (Clover, 1): 9.785417021900077
```

Fig. 2. Top book recommendations obtained for two different users along with predicted ratings.

5 Discussion

Memory-Based Collaborative Filtering (Cosine Similarity):

1. Performance Overview:
 - Both User-Based Cosine Similarity and Item-Based Cosine Similarity models resulted in an RMSE of 7.94, indicating relatively high error rates in predicting ratings. These metrics suggest that the model has a limited ability to accurately predict ratings for unseen users or items.
2. Analysis:
 - The high RMSE indicate that Cosine Similarity-Based models struggle with prediction accuracy, especially when dealing with the cold start problem (e.g., when new items or users are introduced) and sparse data. These memory-based models rely heavily on the historical interaction data between users and items, which limits their ability to generalize well when there is a lack of sufficient data or when faced with new users/items.
 - Additionally, memory-based models are computationally expensive because they compute similarity scores between each pair of users or items, which may not be feasible with large datasets. This increases the overall complexity and can lead to slower performance.
3. Insights:
 - Memory-based models are simple to implement and offer intuitive recommendations based on user/item similarities. However, they are less accurate when compared to model-based methods like SVD because they are based on explicit similarities without considering latent factors that could explain user preferences.
 - As dataset size grows, the scalability of memory-based models becomes a significant limitation. Cosine similarity becomes less effective when there are many users and items, especially if there are few interactions (sparse data).

Model-Based Collaborative Filtering (SVD):

1. Performance Overview:
 - The SVD model achieved a significantly lower RMSE of 1.64, showing a significant improvement over the memory-based methods. These results demonstrate that the model is much better at predicting ratings accurately, capturing the underlying structure in user-item interactions.
2. Analysis:
 - The low RMSE values indicate that the SVD model is much more efficient at predicting ratings for both users and items. By leveraging matrix factorization, SVD reduces the dimensionality of the original

matrix, which helps to capture hidden patterns in user behavior and item characteristics. This is particularly beneficial for sparse data, where many user-item interactions are missing.

- Matrix factorization in SVD allows the system to generalize better to new, unseen users or items because it learns latent factors (hidden features) that explain user's preferences more comprehensively than just user-item similarities.
 - SVD's ability to reduce dimensionality makes it a more scalable solution. Unlike memory-based approaches that require pairwise calculations between all users or items, SVD computes latent factors, making it more computationally efficient in large-scale recommendation systems.
3. Insights:
- The significant improvement in accuracy with SVD supports the idea that model-based methods are generally more robust and scalable, especially in cases where data is sparse. The latent factors that SVD uncovers can capture deeper patterns in the data, leading to better predictions and recommendations.
 - SVD is not only more accurate but also more scalable for large datasets. Its performance remains strong even as the dataset grows, which is crucial for real-world applications with a large number of users and items.
 - Unlike memory-based methods, SVD does not rely on the explicit similarity between users or items. Instead, it finds hidden features, making it less susceptible to issues like cold start (new items or users), and it improves over time as more data is available.

Comparative Analysis:

- Accuracy:
The SVD model outperforms both User-Based and Item-Based Cosine Similarity models by a large margin, achieving an RMSE of 1.64 compared to 7.94 for the cosine similarity models.
- Handling Sparsity:
Memory-based models, including Cosine Similarity, struggle when the user-item interaction matrix is sparse, meaning many ratings are missing. SVD, on the other hand, is designed to deal with such sparsity by capturing latent features and generalizing better to unseen ratings. This makes SVD more suitable for real-world scenarios where data is often incomplete or sparse.
- Scalability:
The Cosine Similarity-based models require calculating pairwise similarities for every user or item, which becomes computationally expensive as the number of users or items grows. On the other hand, SVD reduces the dimensionality of the data, leading to better scalability. This is especially important for systems dealing with large-scale datasets, where SVD can handle more users/items without a significant drop in performance.

6 Challenges and Future Work

6.1 Challenges

1. Sparsity of the Dataset:

One of the primary challenges faced during the project was dealing with the sparsity of the dataset. In real-world scenarios, user-item interaction matrices are often sparse, meaning that most users have only rated a small subset of items. This makes it challenging for traditional recommendation algorithms, especially memory-based collaborative filtering methods (like Cosine Similarity), which rely heavily on available user-item interactions to compute similarities.

2. Cold Start Problem:

The cold start problem arises when new users or new items are added to the system, and there is not enough interaction data available to make accurate recommendations. This problem is particularly evident in memory-based collaborative filtering (user-based or item-based), which requires previous interactions to compute similarities between users or items.

3. Scalability:

While SVD improves accuracy and reduces dimensionality, the training time for matrix factorization methods can still be a challenge when dealing with very large datasets. Even though SVD is more scalable than memory-based methods, for extremely large datasets with millions of users and items, training time and memory requirements can become a concern.

4. Overfitting and Generalization:

Overfitting occurs when the model fits the training data too closely and fails to generalize well to unseen data. Although SVD reduces the dimensionality of the data, there is still the risk of overfitting, especially if the number of latent factors is too high or the regularization is not properly tuned.

6.2 Future Work

6.2.1 Improved Matrix Factorization Techniques:

While SVD is an effective matrix factorization technique, there are other advanced methods, such as SVD++, Non-Negative Matrix Factorization (NMF), or Alternating Least Squares (ALS), that can offer improvements in predictive accuracy and training efficiency. Exploring these more advanced matrix factorization techniques may lead to better handling of sparse data and faster training times, especially for large-scale datasets.

6.2.3 Regularization and Hyperparameter Tuning:

Regularization techniques (such as L2 regularization) can help prevent overfitting by adding a penalty term to the loss function. Proper tuning of the hyperparameters, such as the number of latent factors and regularization strength, can improve the model's generalization ability. Implementing cross-validation and hyperparameter optimization (such as grid search or random search) to find the optimal regularization parameters can help reduce overfitting and improve the model's performance on unseen data.

6.2.3 Better Handling of Cold Start:

Cold start problems could be mitigated by using additional sources of information, such as user demographics, item features, and external data (e.g., social media interactions, reviews, etc.). Content-based methods, which rely on item metadata, can also be further enhanced by using natural language processing (NLP) to extract features from text (e.g., book descriptions, reviews, or authors). Combine collaborative and content-based features in the recommendation engine, utilizing data such as user profiles, social networks, and item content to generate better recommendations for new users and items. Matrix Factorization techniques like SVD++ and TrustSVD can also be used to incorporate trust factor into recommender system.

7 Conclusion

On the Book-Crossing dataset, we developed a recommendation system using collaborative filtering techniques, with a focus on two primary approaches: memory-based collaborative filtering using cosine similarity and model-based collaborative filtering using Singular Value Decomposition (SVD). We used Root Mean Square Error (RMSE) as important measures to compare the performance of different approaches after thorough experimentation and review. The results showed that the SVD-based collaborative filtering strategy outperformed cosine similarity-based models in terms of accuracy and scalability. With an RMSE of 1.64, SVD outperformed the user-based and item-based cosine similarity models, which had an RMSE of 7.94 .

8 References

1. The Book-Crossing Dataset. (2018). *Book-Crossing Dataset*.
2. Park, D. S., Kwon, H., & Kim, J. M. (2012). *A Survey of Collaborative Filtering Techniques*. Proceedings of the International Conference on Machine Learning.
3. Koren, Y., Bell, R., & Volinsky, C. (2009). *Matrix Factorization Techniques for Recommender Systems*. Computer Science.
4. Guibing Guo, Jie Zhang, Neil Yorke-Smith .*TrustSVD: Collaborative Filtering with Both the Explicit and Implicit Influence of User Trust and of Item Ratings*.
5. <https://www.ibm.com/topics/collaborative-filtering>